

CCNA 200-301 V2.0

23

Device Access, AAA and Secure File Transfer

Part IV — Network Services and Security

EXAM TOPICS

4.1

4.2

LECTURER

Dr. Mustafa Hameed



THE ISLAMIA UNIVERSITY OF BAHAWALPUR

Department of Information Technology

Where this chapter sits

Part I	Foundations	chapters 1–7
Part II	Switching and Network Access	chapters 8–14
Part III	IP Routing	chapters 15–19
Part IV	Network Services and Security	chapters 20–26
Part V	Wireless, Virtualisation and Operations	chapters 27–32
Part VI	Sitting the Exam	chapters 33–34

What you will be able to do

- **Configure** local usernames, enable secrets and SSH-only access to the console and VTY lines.
- **Configure** a router or switch as an AAA client against TACACS+ and against RADIUS, with a local fallback that cannot lock you out.
- **Compare** TACACS+ and RADIUS and choose correctly between them.
- **Manage** configuration and image files over SCP and SFTP, and say why v2.0 replaced TFTP with them.

Before we start

Chapter 7 for line configuration and privilege levels. Chapter 21 for `ip domain name`, which SSH key generation requires. Chapter 6 for the difference between TCP and UDP, which is one of the real distinctions between TACACS+ and RADIUS.

Vocabulary for this chapter

AAA

authentication

authorization

accounting

TACACS+

RADIUS

method list

fallback

enable secret

SCP

SFTP

privilege level

Each is defined where it first matters — not here.

Two topics, one chapter, and a v2.0 change worth noticing

This is what the blueprint actually asks for

Topic **4.1** is *configure network devices with local usernames and as an AAA client (TACACS+ and RADIUS) for management*. Topic **4.2** is *manage device configuration and software files using secure file transfer operations with SFTP/SCP*.

They sit together because they are the same job: getting onto a device safely, and moving files onto it safely. And **4.2** is a genuine v2.0 change — v1.1 examined TFTP. TFTP is UDP, has no authentication of any kind, and sends everything in clear text. It has been removed from the blueprint and replaced by two protocols that fix all three problems. If a practice question offers TFTP as the secure answer, the question predates your exam.

Local access done properly



Baseline secure access

IOS · configuration mode

```
hostname CAMPUS-SW1
!  
! A domain name is required before RSA keys can be generated: the key  
! is named host.domain.  
ip domain name campus.example.edu  
!  
! script is type 9. Use it where the platform offers it. Never use  
! "username ... password", which stores type 0 or type 7.  
username netadmin privilege 15 algorithm-type scrypt secret S0me-Long-Passphrase  
username helpdesk privilege 1  algorithm-type scrypt secret An0ther-Passphrase  
!  
enable algorithm-type scrypt secret Enable-Passphrase-Here  
!  
! 2048 is the practical minimum; below 768 the device will refuse to  
! run SSH version 2 at all.  
crypto key generate rsa modulus 2048  
ip ssh version 2  
ip ssh time-out 60  
ip ssh authentication-retries 3
```

Baseline secure access (continued)

IOS · configuration mode

```
!  
line console 0  
  login local  
  exec-timeout 10 0  
  logging synchronous  
!  
line vty 0 15  
  login local  
  transport input ssh  
  exec-timeout 10 0
```

transport input ssh is the line that matters

Generating keys and enabling SSH does not disable Telnet. Until you say `transport input ssh`, the device still accepts plain-text logins on port 23, and an auditor will find it. The default on many platforms is `transport input all`.

The reverse mistake is worse: typing `transport input ssh` *before* the keys exist locks you out of a remote device completely. Generate keys first, prove SSH works from a second window, then restrict the transport.

Password types, and why service password-encryption is not enough

- **Type 0** — plain text. Visible in the configuration.
- **Type 7** — Cisco's reversible obfuscation, applied by `service password-encryption`. It is trivially reversed by tools that have existed for decades. It stops shoulder-surfing and nothing else.

Password types, and why service password-encryption is not enough (continued)

- **Type 5** — salted MD5. A real hash, but MD5 is old.
- **Type 8 (PBKDF2) and type 9 (scrypt)** — current. Use `algorithm-type sha256` or `algorithm-type scrypt`.

So `service password-encryption` is still worth enabling — it covers things that can only be type 7, such as line passwords and TACACS+ keys — but it is not a substitute for storing user passwords as type 8 or 9.

`enable secret` is hashed; `enable password` is not. If both exist, the secret wins and the password is dead configuration that leaks a credential. Delete it.

AAA: three separate questions



AAA

Authentication — who are you? **Authorization** — what are you allowed to do?
Accounting — what did you actually do? Three distinct questions, answered by method lists that a device consults in order.

The distinction that saves you and the one that locks you out

If the TACACS+ server answers “no, wrong password”, that is an answer. The device stops there and rejects you — it does not fall through to the local database. Fallback happens only when the server does not respond at all.

This is correct behaviour and it surprises people. A fallback method is protection against a *dead server*, not against a forgotten password.

AAA against TACACS+, with a local fallback

IOS · configuration mode

```
aaa new-model
!  
! The server. This is the IOS 15 / IOS XE syntax; older configurations  
! used a single "tacacs-server host" line.  
tacacs server TAC1  
  address ipv4 10.0.0.20  
  key SharedSecretHere  
!  
tacacs server TAC2  
  address ipv4 10.0.0.21  
  key SharedSecretHere  
!  
aaa group server tacacs+ TACGROUP  
  server name TAC1  
  server name TAC2  
!  
! Authentication: try the group, then the local database if no server  
! answers. The trailing "local" is not optional in any network you  
! intend to keep working.  
aaa authentication login default group TACGROUP local  
aaa authentication enable default group TACGROUP enable
```

AAA against TACACS+, with a local fallback (continued)

IOS · configuration mode

```
!  
! Authorization: what privilege level does this user get on login?  
aaa authorization exec default group TACGROUP local if-authenticated  
!  
! Accounting: a record of every session and every command at level 15.  
aaa accounting exec default start-stop group TACGROUP  
aaa accounting commands 15 default start-stop group TACGROUP  
!  
! Source the packets from a stable address so the server's client  
! entry always matches, whichever interface the traffic leaves by.  
ip tacacs source-interface Loopback0
```

The same device as a RADIUS client

IOS · configuration mode

```
radius server RAD1
  address ipv4 10.0.0.30 auth-port 1812 acct-port 1813
  key AnotherSharedSecret
!
aaa group server radius RADGROUP
  server name RAD1
!
aaa authentication login default group RADGROUP local
aaa authorization exec default group RADGROUP local
ip radius source-interface Loopback0
```

show aaa servers | include host|state

```
CAMPUS-SW1#
```

```
RADIUS: id 1, priority 1, host 10.0.0.30, auth-port 1812, acct-port 1813  
      State: current UP, duration 43112s, previous duration 0s
```

The one-sentence version

This is what the blueprint actually asks for

TACACS+ manages administrators; RADIUS manages users. If a question is about who may configure a switch, the answer is TACACS+. If it is about a laptop authenticating onto a wireless network, the answer is RADIUS.

Two supporting facts worth memorising because they are asked directly: TACACS+ is TCP 49 and encrypts the whole payload; RADIUS is UDP and encrypts only the password.

Moving files securely



Enabling the SCP server

IOS · configuration mode

*! SCP will not start without all three of these. The authorization
! line in particular is easy to miss, and its absence produces a
! connection that authenticates and then refuses every transfer.*

```
aaa new-model
aaa authentication login default local
aaa authorization exec default local
!
ip scp server enable
```

VERIFY

scp netadmin@10.0.0.1:running-config ./sw1-backup.cfg

Linux or macOS

```
netadmin@10.0.0.1's password:  
running-config
```

```
100% 8241 1.2MB/s 00:00
```

copy sftp: flash:

CAMPUS-SW1#

```
Address or name of remote host []? 10.0.0.40
Source username [netadmin]? images
Source filename []? cat9k_iosxe.17.12.04.SPA.bin
Destination filename [cat9k_iosxe.17.12.04.SPA.bin]?
Accessing sftp://images@10.0.0.40/cat9k_iosxe.17.12.04.SPA.bin...
Loading file ! [OK - 1103237120 bytes]
1103237120 bytes copied in 412.338 secs (2675281 bytes/sec)
```

Backing the configuration up on a schedule

IOS · configuration mode

```
! One-off, non-interactive form. The password appears on the command  
! line, so prefer key-based SCP where the platform supports it.  
! do copy running-config scp://netadmin@10.0.0.40/CAMPUS-SW1.cfg  
!  
! Automatic archive: a copy every time someone saves, kept off-box.  
archive  
  path scp://netadmin@10.0.0.40/archive/$h-  
  write-memory  
  maximum 14
```

Verify the image before you reload onto it

A transfer that reports [OK] has moved the bytes it was given. It has not told you the file is the image Cisco shipped. Before reloading:

- `verify /md5 flash:cat9k_iosxe.17.12.04.SPA.bin` and compare against the hash published with the download.
- `show flash:` to confirm the size matches.
- Confirm free space *before* the copy, not after — a partial copy that filled flash can leave a device unable to boot.

Common pitfalls

- **aaa authentication login default group TACGROUP with no local.** When the server is unreachable, nobody can log in — not even at the console. This is the classic lock-out.
- **Testing AAA by closing your session.** Always open a second session and prove it works before closing the first.
- **Mismatched shared key.** Authentication fails with no useful message on the client. Check the server's logs, not the switch's.
- **No ip tacacs source-interface.** Packets leave by whichever interface routing picks, the server does not recognise the source, and the failure is intermittent by design.
- **transport input ssh before the keys exist.** Instant remote lock-out.

Common pitfalls (continued)

- **Enabling SCP without `aaa authorization exec`.** Logs in, transfers nothing.
- **Believing `service password-encryption` protects anything.** Type 7 is reversible.
- **Reaching for TFTP out of habit.** It is off the blueprint and it was off the good-practice list long before that.

Nobody can log in to any switch. The AAA server is up and other services work.

A campus of forty switches, all authenticating to TACACS+. Overnight, logins begin failing everywhere. The TACACS+ server is running, responds to ping from a workstation, and is authenticating VPN users normally. Console access fails as well as SSH.

Nobody can log in to any switch. The AAA server is up and other services work.

What would you check first — and what do you expect to see?

show run | include aaa authentication

CAMPUS-SW1#

```
aaa authentication login default group TACGROUP local  
aaa authentication enable default group TACGROUP enable
```

show aaa servers | include host|state

CAMPUS-SW1#

```
TACACS+: id 1, priority 1, host 10.0.0.20, auth-port 49  
      State: current UP, duration 61204s
```

What is the fault — and what does this output rule out?

Why it is doing that

Diagnosis. The configuration has a correct fallback and the server is reachable and marked **UP**. That combination rules out both of the usual suspects at once — and it is the clue.

Because the server is reachable, the device sends the request and gets an *answer*. The answer is a rejection. Fallback to **local** never fires, because fallback happens on silence, not on refusal. The fault is therefore on the server: overnight, the group that grants network-device administration was changed, and the server is now correctly refusing everyone.

The tell is that console access fails too. A network-side or transport-side fault would leave the console working, because the console does not depend on the network. A fault that follows you to the console is in the authentication decision itself.

And the lesson

Immediate recovery. You need an account the switch can validate without the server. If a local account exists, use it — but the method list consults TACACS+ first and TACACS+ is answering, so it will not be reached. Break the server's reachability from the switch deliberately, or use the password-recovery procedure on one switch, restore local access, and work from there.

The lasting fix is on the server, not the switches. Restore the administration group.

And the lesson to write down. A local fallback protects you from an unreachable server. It does not protect you from a reachable server that says no. Sites that care about this keep a break-glass local account and test it on a schedule — and keep at least one device out of AAA entirely, reachable out-of-band, as the way back in.

Secure access, AAA, and a backup you can restore

Platform note. Packet Tracer models a simplified AAA server and will carry tasks 1–4. Tasks 5–8 need a real server (`tac_plus` or FreeRADIUS) and a real SSH stack, so use CML or GNS3.

Topology. Two switches and a router, a management VLAN, a Linux host running the AAA server and an SFTP server, and your own workstation.

1. Build baseline access on all three devices: hostname, domain name, two local users at different privilege levels, enable secret, RSA keys, SSH version 2, `transport input ssh`. Verify by SSH and confirm Telnet is refused.
2. Prove the password types. Run `show run | include username` and identify the type number on each. Add one user with `password` instead of `secret` and compare, then remove it.
3. Configure AAA against the server with a local fallback. **Open a second SSH session before you test**, and keep it open throughout.
4. Verify with `show aaa servers` and by logging in as a server-defined user. Confirm the privilege level you land in.

Secure access, AAA, and a backup you can restore (continued)

5. **Break 1.** Stop the AAA server. Log in again and confirm the local fallback works. Record how long the device waits before falling back.
6. **Break 2.** Restart the server and disable your test account on it. Attempt to log in. Explain why the local fallback does *not* rescue you, and relate this to the Break & Fix above.
7. **Break 3.** Remove `local` from the method list, stop the server, and log out of every session. Recover the device. Write down how long it took and what you would have done differently.
8. Enable the SCP server and copy the running configuration to your workstation. Then omit `aaa authorization exec` and observe the different failure.

Secure access, AAA, and a backup you can restore (continued)

9. Configure `archive` to write to SFTP on every `write memory`. Make a change, save, and confirm the archived file appears.
10. **Extension.** Copy an image by SFTP, verify its MD5 against the published hash, and deliberately truncate a copy to see what `verify` reports.

CHECKPOINT 1 of 5

In aaa authentication login default group TACGROUP local, when is local used and when is it not?

Discuss · then advance

Checkpoint 1 of 5

In aaa authentication login default group TACGROUP local, when is local used and when is it not?

`local` is used only when the TACACS+ group *does not respond at all*. If a server answers and rejects the credentials, that is an answer, the list ends, and the local database is never consulted. Fallback protects against a dead server, not a forgotten password.

CHECKPOINT 2 of 5

Give the transport, port and encryption scope of TACACS+ and of RADIUS, and say which you would choose for controlling switch administrators.

Discuss · then advance

Checkpoint 2 of 5

Give the transport, port and encryption scope of TACACS+ and of RADIUS, and say which you would choose for controlling switch administrators.

TACACS+ is TCP 49 and encrypts the entire packet body; RADIUS is UDP 1812/1813 and encrypts only the password field. Choose **TACACS+** for switch administrators — it separates authorization fully and supports per-command control.

CHECKPOINT 3 of 5

Why must ip domain name be set before generating RSA keys?

Discuss · then advance

Checkpoint 3 of 5

Why must `ip domain name` be set before generating RSA keys?

Because the RSA key pair is named `hostname.domain`. Without a domain name the device has no full name to label the key with and refuses.

CHECKPOINT 4 of 5

What three commands does the IOS SCP server require, and what is the symptom when the authorization line is missing?

Discuss · then advance

Checkpoint 4 of 5

What three commands does the IOS SCP server require, and what is the symptom when the authorization line is missing?

`aaa new-model`, an `aaa authentication login` method, and `aaa authorization exec` — plus `ip scp server enable` itself. Without the authorization line, users authenticate successfully and every transfer is then refused.

CHECKPOINT 5 of 5

Why did v2.0 replace TFTP with SFTP and SCP? Give three reasons.

Discuss · then advance

Checkpoint 5 of 5

Why did v2.0 replace TFTP with SFTP and SCP? Give three reasons.

TFTP has no user authentication, sends everything in clear text, and runs over UDP with no integrity protection. A configuration file in flight — including its shared secrets — is readable by anyone on the path, and anyone who can reach the server can fetch any file whose name they guess.

What to take away

- Baseline first: hostname, domain name, local users with `algorithm-type` `scrypt` `secret`, `enable` `secret`, RSA keys, `ip` `ssh` `version` `2`, `login` `local`, `transport` `input` `ssh`.
- Type 7 is obfuscation, not encryption. Type 8 and 9 are the current choices. `enable` `password` should not exist.
- AAA answers three separate questions through ordered method lists. A method is skipped only when it does not respond — a rejection ends the list.
- Always end an authentication method list with `local`, and always test from a second session.

What to take away (continued)

- TACACS+: TCP 49, whole packet encrypted, full AAA separation, per-command authorization, device administration. RADIUS: UDP 1812/1813, password only encrypted, combined authentication and authorization, network access.
- SCP puts file transfer inside SSH; IOS can be an SCP server and an SFTP client. Both replace TFTP, which authenticates nobody and encrypts nothing.
- Verify an image's hash before reloading onto it.

Where we got to

- Baseline first: `hostname`, domain name, local users with `algorithm-type` `scrypt` `secret`, `enable` `secret`, RSA keys, `ip ssh version 2`, `login local`, `transport input ssh`.
- Type 7 is obfuscation, not encryption. Type 8 and 9 are the current choices. `enable password` should not exist.
- AAA answers three separate questions through ordered method lists. A method is skipped only when it does not respond — a rejection ends the list.
- Always end an authentication method list with `local`, and always test from a second session.

NEXT

Chapter 24 — Access Control Lists